Simon Chen

347-322-4133 | shangminch@gmail.com | simon-chen.com | linkedin.com/in/shangmin-chen | github.com/shangmin-chen

Education

Boston University Boston, MA

Bachelor of Arts in Computer Science *August 2026*

- Relevant Coursework: Data Structures & Algorithms, Operating Systems, Distributed Systems, Databases, Machine Learning, Data Science.

Experience

Software Engineering Intern February 2026 – June 2026

RESET Standard *Shanghai, China*

- Engineered asynchronous data pipelines with RabbitMQ, Redis, and Sidekiq to process long-running ingestion jobs in the background, decoupling external API calls from user-facing requests.
- Doubled the platform's supported environmental data categories (from 2 to 4) by independently implementing end-to-end water and waste data integration, developing Rails backend ingestion pipelines, PostgreSQL schemas, and client-facing React/Vite/Tailwind dashboards.
- Redesigned project-device mapping workflows in response to client requests, refactoring legacy forms to streamline onboarding of environmental monitoring devices.

Lead Software Engineer December 2025 – Present

EZ Esports *New York, NY*

- Designed a PostgreSQL schema on Supabase using Drizzle ORM, supporting 500+ matches across 20+ teams, and built RESTful endpoints for player statistics, standings, and historical data.
- Cut page load time by 75% (from 3.2s to 0.8s) by implementing edge caching on Cloudflare, route-based code splitting, and asset optimization for the Next.js frontend.
- Migrated an esports league platform from Google Sites to Next.js with a Supabase backend, replacing manual content updates with live scoring and automated season management.

Projects

Persephone | *Python, Cython/C++17, Rust, Modal, FAISS* May 2026

- Built and operated a live Kalshi BTC prediction-market trading system end to end (market-data ingestion, theo serving, strategy evaluation, execution, and risk), generating $8K net PnL on $200K traded volume in its first 4 days.
- Rewrote 9 live hot-path kernels (order-book maintenance, order-book imbalance, trade intensity, event aggregation) as native Cython extensions, cutting Kalshi book maintenance from 21.8ms to 195μs p50 (112x).
- Designed a lock-free SPSC event ring in Cython/C++17 with cache-line-aligned atomic cursors and packed uint64 records, benchmarked at 584ns per enqueue and 703ns per event drained.
- Replaced KDE inference with a GBM theo layer, cutting model query latency from $\sim$1.5ms to $\sim$20μs per query ($\sim$75x) behind an unchanged live serving interface.
- Prevented bad-price quoting with fail-closed gates that block stale, crossed, or untrusted market data before quote generation, computing fused market metrics in 357μs p50 across 8 venues.
- Eliminated duplicate-send and crash-replay risk with a durable-before-wire order layer: intent journaled before dispatch, deterministic client IDs for idempotent retries, and persisted ack state for reconciliation.

Hermes Letters | *Next.js, TypeScript, Supabase, Drizzle ORM, PostgreSQL* June 2026

- Ensured consistent letter claims via atomic SQL updates and secured sessions in XSS-resistant httpOnly cookies.
- Derived user connections at query time from kept-letter records using indexed lookups, avoiding a separate relationship table and associated database sync issues.
- Hardened media storage by validating uploads via server-side magic-byte signatures, storing files in a private Supabase bucket, and restricting delivery to 1-hour signed URLs.

Technical Skills

Languages: C++17, Python, Cython, Java, TypeScript, JavaScript, Rust, SQL
Frameworks & Libraries: Spring Boot, FastAPI, React, Next.js, FAISS, CuPy, Tailwind CSS
Databases & Infrastructure: PostgreSQL, Redis, RabbitMQ, Docker, Nginx, Modal, OpenMP